

Windows Endpoint Persistence Investigation

Scenario: A suspicious executable named challenge.exe has been executed on a Windows workstation. Your task is to identify listening network activity and persistence mechanisms created by the binary using endpoint analysis commands.

Lab safety note: This is a controlled classroom challenge. Run it only inside the approved lab VM as discussed during the lecture. Keep the challenge window open until the investigation is complete, then use the revert command.

Student task: Complete each question. For each answer, submit the command used, the value found, and a brief justification explaining how the command supports your answer.

Start the Challenge: Open Command Prompt as Administrator. Navigate to the folder containing challenge.exe, then execute it. **Leave the window open until the investigation is complete.**

```
challenge.exe
```

Open a second Administrator PowerShell window for the investigation. Do not run the investigation in the same window where challenge.exe is running.

Start the challenge - Confirm that challenge.exe has been executed and is still running.

```
Get-Process -Name challenge
```

Identify the malware process ID: Find the PID of challenge.exe.

```
Get-Process -Name challenge
```

```
Get-Process -Name challenge | Select-Object Id,ProcessName,Path
```

Identify the listening port - Find the local TCP port opened by challenge.exe.

```
Get-NetTCPConnection -State Listen | Where-Object {$_.OwningProcess -eq <PID>}
```

Identify the executable path: Determine the full path from which challenge.exe is running.

```
Get-Process -Name challenge | Select-Object Id,ProcessName,Path
```

List DLL modules beginning with m: List loaded modules for challenge.exe and identify the two DLLs whose names begin with m.

```
(Get-Process -Id <PID>).Modules | Select-Object ModuleName,FileName
```

```
(Get-Process -Id <PID>).Modules | Where-Object {$_.ModuleName -like "m*.dll"} | Select-Object ModuleName
```

```
tasklist /m /fi "PID eq <PID>"
```

Identify the parent process: Determine which process launched challenge.exe.

```
Get-CimInstance Win32_Process -Filter "name='challenge.exe'" | Select-Object ProcessId,ParentProcessId,CommandLine
```

```
Get-Process -Id <ParentPID>
```

Find the attacker-created SMB share: List local SMB shares and identify the non-default share created by the binary.

```
Get-SmbShare
```

```
net share
```

```
Get-SmbShare | Where-Object {$_.Name -notin "ADMIN$","C$","IPC$","print$"}
```

Identify the SMB share path: Determine the local folder path exposed by the attacker-created share. Replace <ShareName> with the suspicious share name.

```
Get-SmbShare -Name "<ShareName>" | Select-Object Name,Path
```

Identify the Registry Run key path: Find the Registry Run key used by the binary for user logon persistence. Look for a suspicious value pointing to a user-writable path such as Downloads, Documents, AppData, or Temp.

```
Get-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"
```

```
Get-ItemProperty "HKLM:\Software\Microsoft\Windows\CurrentVersion\Run"
```

Identify the malicious Run entry name: Find the name of the suspicious Registry value in the Run key.

```
Get-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"
```

```
Get-ItemProperty "HKLM:\Software\Microsoft\Windows\CurrentVersion\Run"
```

Identify the Run entry image path: Find the executable path configured in the suspicious Run key value. Replace <EntryName> with the suspicious value name.

```
(Get-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run").<EntryName>
```

Identify the backdoor service name: Find the suspicious Windows service installed by the binary. Some ProgramData services may be legitimate. Focus on unusual services pointing to user folders.

```
Get-CimInstance Win32_Service | Where-Object {$_.PathName -match  
"Users|AppData|Temp|Downloads|Desktop|ProgramData"} | Select-Object  
Name,DisplayName,State,StartMode,PathName
```

Identify the service start type: Determine whether the suspicious service is set to start automatically or manually. Use sc.exe in PowerShell, not sc alone.

```
Get-CimInstance Win32_Service -Filter "Name='<ServiceName>'" | Select-Object  
Name,StartMode
```

```
sc.exe qc <ServiceName>
```

Identify the service binary path: Find the executable path configured as the service binary.

```
Get-CimInstance Win32_Service -Filter "Name='<ServiceName>'" | Select-Object  
Name,PathName
```

Identify the attacker-created scheduled task: Find the scheduled task created by the binary. Ignore known legitimate Windows Defender tasks under C:\ProgramData\Microsoft\Windows Defender.

```
Get-ScheduledTask | Select-Object TaskName,TaskPath,State
```

Identify the scheduled task executable: Determine the executable launched by the suspicious scheduled task.

```
(Get-ScheduledTask -TaskName "<TaskName>").Actions
```

Identify the scheduled task run time: Determine when the scheduled task is configured to run. Look for StartBoundary or Start Time.

```
(Get-ScheduledTask -TaskName "<TaskName>").Triggers  
  
schtasks /query /tn "<TaskName>" /v /fo list
```

Clean up and revert the system: Terminate challenge.exe and restore the system to its original state. Enter done after cleanup is complete.

```
# In the original challenge window, press Ctrl + C  
  
challenge.exe -revert  
  
Get-Process -Name challenge -ErrorAction SilentlyContinue
```

Submission Requirements

- Answers to all questions.
- The command used for each technical answer.
- A brief explanation of the evidence.
- Screenshots are optional unless requested by the instructor.

Answer Key

The answers below are based on the specific execution output provided for this lab run.

Question	Finding	Expected answer
1	Challenge started	done
2	PID of challenge.exe	8772
3	Listening port	50050
4	Full path of challenge.exe	C:\Users\MA-Sandbox\Desktop\COD Lab - Tools and Samples\Endpoint Security\challenge.exe
5	DLLs beginning with m	msvcrt.dll, mswsock.dll
6	Parent process	cmd.exe
7	Attacker-created SMB share	xkalibur
8	Share path	C:\Users\CDO\AppData\Local\Temp\46d5b8556d0d3e30ec1
9	Registry Run key path	HKCU\Software\Microsoft\Windows\CurrentVersion\Run
10	Malicious Run entry name	CleanUpController
11	Run key image path	C:\Users\CDO\Downloads\wininit.exe
12	Backdoor service name	WindowsActiveService
13	Service start type	Auto / AUTO_START
14	Service binary path	C:\Users\CDO\Documents\svcbackdoor.exe
15	Scheduled task name	ayttnzc
16	Scheduled task executable	C:\Users\CDO\Downloads\beac0n.exe
17	Scheduled task run time	03:30:00 / 3:30 AM
18	Cleanup completed	done

Explanation of challenge.exe: Based on the observed artifacts, challenge.exe is a controlled malware-simulation binary for endpoint investigation training. When executed with administrator privileges, it opens a listening TCP socket, creates an attacker-controlled SMB share, adds Registry Run key persistence, installs a backdoor-like Windows service, and creates a scheduled task that launches a suspicious executable. It behaves like a post-compromise persistence simulator designed to help SOC analysts practice identifying Windows persistence and staging artifacts.